

MAULAS 3.0

Auditoría Técnica, Arquitectura y Plan Director

VOLUMEN I

Arquitectura General y Estado del Proyecto

Versión: 1.0

Destinatario: Anti Gravity

Estado: Documento técnico consolidado

Autor de la auditoría: ChatGPT (OpenAI)

1. Resumen ejecutivo

1.1 Objetivo

El objetivo de esta auditoría es evaluar la arquitectura actual de la aplicación **Maulas**, identificar sus fortalezas y debilidades desde un punto de vista técnico y proponer una hoja de ruta para su evolución sin comprometer la estabilidad del proyecto.

Este documento no pretende rediseñar la aplicación desde cero ni imponer una tecnología concreta. Su finalidad es proporcionar un análisis arquitectónico que permita tomar decisiones fundamentadas sobre la evolución del software.

1.2 Alcance

La auditoría se ha realizado sobre el código fuente del proyecto y sobre la estructura general de la aplicación.

El análisis ha cubierto, entre otros aspectos:

- Arquitectura general.
- Organización del código.
- Separación de responsabilidades.
- Modularidad.
- Flujo de datos.
- Uso de Firebase.
- Organización de JavaScript.
- Escalabilidad.

- Mantenibilidad.
- Riesgos técnicos.
- Posibilidades de optimización.

No se han realizado pruebas de carga ni mediciones de rendimiento en ejecución.

Por tanto, todas las afirmaciones relacionadas con tiempos de ejecución, consumo de memoria o velocidad deben entenderse como hipótesis razonables derivadas de la inspección del código y deberán validarse mediante herramientas de perfilado.

2. Resumen de conclusiones

Tras el análisis completo del proyecto, la conclusión principal es la siguiente:

Maulas no necesita una reescritura. Necesita una evolución arquitectónica.

Esta conclusión es especialmente relevante.

Durante la revisión del proyecto se consideraron distintas posibilidades, incluida una migración hacia un framework moderno.

Tras finalizar el análisis completo, esa opción dejó de considerarse prioritaria.

La tecnología utilizada actualmente es perfectamente válida para soportar la evolución del proyecto siempre que se refuerce la arquitectura interna.

Las principales conclusiones son:

- La base del proyecto es sólida.
- La organización general es mejor de lo esperado.
- Existe una separación razonable por funcionalidades.
- El código ha crecido más rápido que la arquitectura.
- Los principales problemas son arquitectónicos, no tecnológicos.
- El mayor retorno de inversión vendrá de reorganizar la aplicación, no de cambiar de framework.

3. Estado actual del proyecto

Desde un punto de vista arquitectónico, Maulas puede clasificarse como una aplicación web de complejidad media-alta.

No se trata ya de una colección de páginas independientes.

Existe una lógica de negocio importante relacionada con:

- usuarios
- jornadas
- pronósticos
- clasificación
- estadísticas
- bote
- administración
- autenticación
- configuración

Esto convierte a Maulas en un producto software con necesidades arquitectónicas distintas a las de una página web convencional.

3.1 Nivel de madurez

La aplicación ha superado claramente la fase de prototipo.

Presenta características propias de un producto consolidado:

- autenticación
- persistencia
- módulos diferenciados
- configuración
- administración
- evolución continuada

Este nivel de madurez implica que las decisiones de arquitectura tienen un impacto mayor que la incorporación de nuevas funcionalidades.

4. Fortalezas detectadas

Durante la auditoría se identificaron numerosos aspectos positivos.

4.1 Organización general

La estructura del proyecto resulta coherente.

Los distintos módulos mantienen una separación funcional razonable.

No se aprecia un crecimiento completamente descontrolado del código.

4.2 Modularidad

Aunque existen módulos de gran tamaño, la aplicación ya presenta una cierta organización por responsabilidades.

Esta característica facilitará una futura refactorización incremental.

4.3 Tecnología

No se ha encontrado ninguna limitación importante derivada del uso de:

- JavaScript moderno
- Firebase
- GitHub Pages

Ninguna de estas tecnologías constituye actualmente el principal cuello de botella del proyecto.

4.4 Escalabilidad potencial

La aplicación puede seguir creciendo.

Sin embargo, ese crecimiento debería apoyarse en una mejora previa de la arquitectura.

El problema no es la capacidad de la tecnología utilizada.

El problema es la organización interna del proyecto.

5. Debilidades detectadas

La auditoría identifica cinco áreas principales de mejora.

5.1 Estado distribuido

La aplicación carece de una fuente única de verdad para representar el estado global.

Esto dificulta:

- reutilizar información
 - sincronizar módulos
 - reducir cálculos repetidos
 - simplificar el mantenimiento
-

5.2 Responsabilidades mezcladas

Algunos módulos combinan:

- acceso a datos
- lógica de negocio
- cálculos
- manipulación del DOM
- gestión de eventos

Esta concentración de responsabilidades incrementa la complejidad del mantenimiento.

5.3 Crecimiento desigual

El proyecto ha incorporado nuevas funcionalidades durante un largo periodo.

Como consecuencia, algunos módulos han crecido considerablemente mientras otros permanecen pequeños.

Esta asimetría dificulta la comprensión global del sistema.

5.4 Reutilización limitada

Existen oportunidades para reutilizar mejor:

- componentes
- cálculos
- consultas

- estructuras de datos

No se trata de un problema grave, pero sí representa una oportunidad clara de mejora.

5.5 Arquitectura evolutiva

La arquitectura original fue adecuada para el tamaño inicial del proyecto.

Sin embargo, el crecimiento funcional hace recomendable introducir una nueva capa arquitectónica que permita seguir evolucionando sin incrementar la complejidad.

6. Conclusión del Volumen I

La auditoría permite afirmar que Maulas dispone de una base técnica suficientemente sólida como para evolucionar durante los próximos años sin necesidad de una reconstrucción completa.

Las mejoras prioritarias no consisten en sustituir tecnologías existentes, sino en reforzar la organización interna del software.

Las decisiones de mayor impacto identificadas en esta fase son:

- introducir una gestión centralizada del estado;
- desacoplar completamente la lógica de negocio de la presentación;
- reorganizar los módulos con mayor responsabilidad;
- preparar la arquitectura para un crecimiento sostenido.

Estas conclusiones constituyen el punto de partida para el resto de volúmenes de esta auditoría, donde se analizarán en profundidad el flujo de datos, la organización del código, el rendimiento, Firebase y la arquitectura propuesta para Maulas 3.0.

FIN DEL VOLUMEN I (Parte 1).

MAULAS 3.0

Auditoría Técnica, Arquitectura y Plan Director

VOLUMEN I (Continuación)

7. Metodología de la auditoría

7.1 Enfoque utilizado

La auditoría se ha realizado siguiendo un enfoque de revisión arquitectónica estática.

El objetivo no ha sido detectar errores funcionales concretos, sino evaluar si la estructura del software permitirá que el proyecto continúe creciendo sin que el coste de mantenimiento aumente de forma desproporcionada.

Durante el análisis se han revisado los siguientes aspectos:

- Organización del proyecto.
- Distribución de responsabilidades.
- Estructura de módulos.
- Dependencias entre componentes.
- Flujo de datos.
- Organización del código JavaScript.
- Gestión de Firebase.
- Posibles cuellos de botella arquitectónicos.
- Escalabilidad.
- Mantenibilidad.
- Riesgos técnicos.

No se han realizado pruebas de carga ni benchmarking.

Por ello, cuando una recomendación dependa de medidas objetivas de rendimiento, se indicará expresamente que requiere validación mediante perfilado.

8. Criterios de evaluación

La valoración del proyecto se ha realizado utilizando ocho criterios principales.

Arquitectura

Se ha analizado si la organización general facilita la evolución futura del software.

No se ha valorado únicamente si "funciona", sino si seguirá siendo mantenible cuando el proyecto duplique su tamaño.

Cohesión

Cada módulo debería tener una responsabilidad claramente definida.

Cuando un mismo archivo gestiona datos, cálculos, interfaz y eventos simultáneamente, la cohesión disminuye.

Este aspecto será tratado en profundidad en el Volumen II.

Acoplamiento

Se ha estudiado el grado de dependencia entre módulos.

Cuanto menor sea el conocimiento que unos módulos tengan sobre otros, más sencilla resultará la evolución del proyecto.

Escalabilidad

Se ha evaluado la capacidad de incorporar nuevas funcionalidades sin modificar grandes cantidades de código existente.

Reutilización

Se ha analizado si la lógica existente puede reutilizarse fácilmente desde distintas partes de la aplicación.

Rendimiento potencial

No se ha medido el rendimiento real.

Se han identificado únicamente patrones que, por experiencia, suelen producir trabajo innecesario para el navegador.

Mantenibilidad

Se ha valorado el esfuerzo esperado para introducir cambios futuros.

Riesgo técnico

Se ha estimado el impacto que tendría continuar desarrollando nuevas funcionalidades sin realizar previamente una evolución arquitectónica.

9. Valoración global

La puntuación obtenida por la aplicación es la siguiente.

Área	Valoración
Organización	9/10
Modularidad	8,5/10
Legibilidad	8,5/10
Escalabilidad actual	7,5/10
Escalabilidad potencial	9,5/10
Mantenibilidad	8,5/10
Arquitectura	8,5/10

La puntuación anterior no pretende comparar Maulas con productos comerciales de gran escala.

Se refiere al estado del proyecto teniendo en cuenta su tamaño, objetivos y evolución.

10. Qué problemas NO tiene Maulas

Una parte importante de cualquier auditoría consiste en identificar aquello que no requiere cambios.

En este proyecto conviene evitar actuaciones innecesarias.

No existe necesidad de una reescritura completa

Durante el análisis no se ha encontrado ninguna evidencia que justifique reiniciar el proyecto desde cero.

Una reescritura implicaría un elevado coste con un beneficio limitado.

No existe necesidad inmediata de cambiar de framework

En fases iniciales de la auditoría se consideró esta posibilidad.

Tras revisar la estructura completa del proyecto, esa recomendación fue descartada.

Los problemas identificados no derivan del uso de JavaScript sin framework.

Derivan de la organización interna de la aplicación.

Firebase no constituye el problema principal

Firebase presenta limitaciones conocidas.

Sin embargo, en el contexto actual del proyecto no representa el principal cuello de botella.

La prioridad debe centrarse en mejorar la forma en que la aplicación consume los datos.

GitHub Pages tampoco limita la evolución

La plataforma de despliegue resulta suficiente para el estado actual del proyecto.

No se considera prioritario migrar el alojamiento.

11. Qué problemas SÍ deben abordarse

Aunque la base sea sólida, existen aspectos cuya mejora tendrá un retorno elevado.

Los más importantes son:

- incorporación de un estado global;
- reorganización de módulos con exceso de responsabilidades;
- separación estricta entre presentación y lógica de negocio;
- centralización del acceso a datos;
- reducción del trabajo repetitivo durante el renderizado.

Estas actuaciones constituyen el núcleo de la propuesta Maulas 3.0.

12. Riesgo de no actuar

Si el proyecto continúa creciendo manteniendo la arquitectura actual, es razonable esperar:

- incremento del tiempo necesario para implementar nuevas funciones;
- mayor probabilidad de introducir regresiones;
- aumento del tamaño de los módulos principales;
- mayor complejidad para localizar errores;
- duplicación progresiva de lógica.

No se trata de problemas inmediatos.

Son riesgos asociados al crecimiento natural del software.

Precisamente por ello resulta recomendable actuar antes de que el tamaño del proyecto se duplique nuevamente.

13. Cierre del Volumen I

La principal conclusión obtenida durante esta fase puede resumirse en una única idea:

La tecnología utilizada por Mautas es adecuada. La prioridad no debe ser cambiar la tecnología, sino fortalecer la arquitectura que la sustenta.

Esta conclusión condiciona el resto de recomendaciones del informe.

En los siguientes volúmenes no se propondrán reescrituras completas, sino una evolución incremental basada en principios de ingeniería del software, minimizando riesgos y preservando el conocimiento ya incorporado al proyecto.

FIN DEL VOLUMEN I

Continúa

MAULAS 3.0

Auditoría Técnica, Arquitectura y Plan Director

VOLUMEN II

Organización del Código y Modularidad

1. Introducción

La calidad de una aplicación no depende únicamente de que funcione correctamente. A medida que un proyecto crece, la organización del código pasa a ser uno de los factores que más condicionan la velocidad de desarrollo, la aparición de errores y la facilidad para incorporar nuevas funcionalidades.

Durante esta fase se ha analizado la organización interna del proyecto con el objetivo de determinar si la estructura actual facilitará la evolución de Maulas durante los próximos años.

La conclusión general es positiva.

El proyecto presenta una organización superior a la que suele encontrarse en aplicaciones personales de tamaño similar. Sin embargo, también se observan síntomas claros de crecimiento orgánico que aconsejan una reorganización antes de seguir aumentando la funcionalidad.

2. Estado actual

La aplicación se encuentra organizada mediante módulos JavaScript relativamente independientes.

Esta aproximación ha permitido mantener un cierto orden conforme el proyecto ha ido creciendo.

Sin embargo, la organización actual está basada principalmente en **pantallas** y no en **dominios funcionales**.

Es decir, existen módulos asociados a páginas concretas que terminan acumulando responsabilidades muy diferentes.

Este patrón suele funcionar correctamente durante las primeras fases de un proyecto, pero comienza a mostrar limitaciones cuando la lógica de negocio aumenta.

3. Evaluación de la modularidad

La modularidad del proyecto puede calificarse como **buena**.

Se aprecia un esfuerzo por separar funcionalidades y evitar concentrar todo el código en un único archivo.

No obstante, la modularidad actual presenta diferencias importantes entre unos módulos y otros.

Mientras algunos mantienen una única responsabilidad relativamente clara, otros han evolucionado hasta convertirse en verdaderos "centros de operaciones" que gestionan simultáneamente:

- acceso a datos;
- lógica de negocio;
- cálculos;
- manipulación del DOM;
- eventos de usuario;
- inicialización de la página.

Este crecimiento desigual constituye el principal foco de deuda técnica identificado en esta fase.

4. Análisis de responsabilidades

Uno de los principios fundamentales de la ingeniería del software es el **Principio de Responsabilidad Única (Single Responsibility Principle)**.

Aplicado a Mauias significa que cada módulo debería tener un único motivo para cambiar.

Durante la revisión se observó que algunos archivos ya no cumplen completamente este principio.

No implica que estén mal implementados.

Significa que han ido incorporando nuevas funciones a lo largo del tiempo hasta convertirse en módulos demasiado versátiles.

Esta situación incrementa:

- el riesgo de introducir errores;
- la dificultad para realizar pruebas;

- el tiempo necesario para comprender el funcionamiento del código;
 - el coste de mantenimiento.
-

5. Módulos candidatos a refactorización

Los módulos que presentan mayor potencial de mejora son aquellos que concentran un elevado número de responsabilidades.

Entre ellos destacan especialmente:

- **bote.js**
- **pronosticos.js**

Es importante destacar que esta recomendación **no implica que dichos módulos estén mal diseñados**.

Al contrario.

Su crecimiento suele ser consecuencia de haber centralizado correctamente funcionalidades relacionadas.

Precisamente por ello resulta recomendable dividirlos ahora que todavía es posible hacerlo de forma controlada.

6. Estrategia de reorganización

La auditoría propone abandonar progresivamente una organización basada en páginas para evolucionar hacia una organización basada en dominios funcionales.

Ejemplo:

Arquitectura actual (simplificada)

dashboard.js

pronosticos.js

bote.js

usuarios.js

Arquitectura propuesta

usuarios/

└─ usuarios-service.js

└─ usuarios-repository.js

- └─ usuarios-view.js
- └─ usuarios-events.js

pronosticos/

- └─ pronosticos-service.js
- └─ pronosticos-repository.js
- └─ pronosticos-renderer.js
- └─ pronosticos-events.js

bote/

- └─ bote-service.js
- └─ bote-calculator.js
- └─ bote-renderer.js
- └─ bote-events.js

Este cambio no modifica la funcionalidad de la aplicación.

Únicamente redistribuye responsabilidades.

7. Separación recomendada

Cada dominio debería contener, como mínimo, las siguientes capas.

Repository

Responsable exclusivamente del acceso a Firebase.

Nunca contiene lógica de negocio.

Service

Implementa las reglas del negocio.

Realiza cálculos.

Coordina operaciones.

No modifica directamente la interfaz.

Renderer

Responsable únicamente de actualizar la interfaz.

No realiza cálculos complejos.

No consulta Firebase.

Events

Gestiona exclusivamente la interacción del usuario.

No contiene lógica de negocio.

8. Beneficios esperados

La reorganización propuesta aporta ventajas importantes.

Reducción del acoplamiento

Cada módulo conocerá únicamente aquello que realmente necesita.

Mayor reutilización

La lógica podrá utilizarse desde distintas pantallas sin duplicar código.

Mejor mantenibilidad

Los cambios futuros afectarán a un número mucho menor de archivos.

Mayor facilidad para realizar pruebas

Las funciones de negocio podrán probarse independientemente de la interfaz.

Incorporación más sencilla de nuevas funcionalidades

El crecimiento dejará de concentrarse siempre sobre los mismos archivos.

9. Riesgos de la reorganización

La refactorización también presenta riesgos que conviene controlar.

Los principales son:

- romper dependencias implícitas;
- mover responsabilidades sin actualizar referencias;
- introducir duplicidades temporales.

Por ello se recomienda realizar la migración de forma incremental.

Nunca mediante una reescritura completa.

10. Recomendaciones para Anti Gravity

Las actuaciones prioritarias recomendadas son:

R-01

Identificar módulos con múltiples responsabilidades.

Prioridad: Muy Alta.

R-02

Separar acceso a datos, lógica y presentación.

Prioridad: Muy Alta.

R-03

Adoptar organización por dominios.

Prioridad: Alta.

R-04

Crear componentes reutilizables.

Prioridad: Alta.

R-05

Evitar que las vistas contengan lógica de negocio.

Prioridad: Muy Alta.

11. Conclusiones

La organización actual del código constituye una buena base para continuar desarrollando Mautas.

Sin embargo, el crecimiento continuado del proyecto hace recomendable una reorganización progresiva antes de seguir incorporando funcionalidades de gran tamaño.

La auditoría concluye que la principal necesidad no consiste en escribir menos código, sino en distribuirlo mejor.

La modularización propuesta permitirá reducir la deuda técnica acumulada, facilitar el mantenimiento y preparar la aplicación para la siguiente etapa de evolución sin necesidad de modificar la tecnología utilizada.

FIN DEL VOLUMEN II

MAULAS 3.0

Auditoría Técnica, Arquitectura y Plan Director

VOLUMEN III

Flujo de Datos, Gestión del Estado y Comunicación entre Módulos

1. Introducción

Una de las conclusiones más importantes obtenidas durante la auditoría es que el principal factor limitante para la evolución de Maulas no es la tecnología utilizada, sino la forma en que circula la información por la aplicación.

La gestión del estado constituye el núcleo de cualquier aplicación moderna.

Cuando el número de funcionalidades crece, disponer de una única representación coherente del estado deja de ser una mejora para convertirse en una necesidad arquitectónica.

Tras revisar el proyecto completo, ésta se considera la actuación con mayor impacto sobre la mantenibilidad, la escalabilidad y el rendimiento futuro.

2. Situación actual

Actualmente la información fluye entre distintos módulos de forma funcional, pero distribuida.

Cada pantalla obtiene los datos que necesita, realiza transformaciones, calcula resultados y actualiza la interfaz.

Este enfoque funciona correctamente mientras el tamaño del proyecto permanece contenido.

Sin embargo, conforme aumenta el número de pantallas aparecen varios efectos secundarios:

- repetición de cálculos;
- duplicación de consultas;
- lógica distribuida;
- mayor acoplamiento entre módulos;
- dificultad para reutilizar información.

No se trata de errores de implementación.

Es una consecuencia natural del crecimiento del software.

3. Problema identificado

La aplicación no dispone de una **fuentes única de verdad**.

En otras palabras, el estado del sistema no se encuentra representado mediante una estructura centralizada desde la que todos los módulos obtengan información.

Esto provoca que diferentes partes de la aplicación necesiten reconstruir o reinterpretar los mismos datos.

A medida que el proyecto crezca, esta situación tenderá a incrementar la complejidad.

4. Principio arquitectónico recomendado

La auditoría recomienda adoptar el siguiente principio:

Toda la aplicación debe leer el estado desde un único punto.

Ese punto recibe habitualmente el nombre de **AppState**.

No constituye una tecnología.

No depende de React, Vue o Angular.

Es simplemente una decisión de arquitectura.

5. Propuesta de AppState

La aplicación debería disponer de un objeto global que represente el estado completo del sistema.

Ejemplo conceptual:

AppState

|

└─ usuarioActual

└─ usuarios

└─ temporada

└─ jornadas

- |— partidos
- |— pronosticos
- |— clasificacion
- |— bote
- |— estadisticas
- |— configuracion
- |— permisos
- |— tema
- |— estadoConexion

Esta estructura no implica que toda la información permanezca permanentemente en memoria.

Representa únicamente la organización conceptual del estado.

6. Beneficios

La incorporación de un AppState produciría mejoras inmediatas.

Consistencia

Toda la aplicación consultará exactamente la misma información.

Reducción de duplicidades

Los cálculos dejarán de repetirse innecesariamente.

Mayor reutilización

Las nuevas pantallas podrán utilizar información ya disponible.

Simplificación del mantenimiento

Será mucho más sencillo localizar el origen de un problema.

Preparación para futuras mejoras

La incorporación de:

- caché;
- modo offline;
- sincronización;
- renderizado incremental;

resultará considerablemente más sencilla.

7. Separación entre datos y presentación

Durante la auditoría se observó otro aspecto importante.

Las pantallas no deberían conocer cómo se obtienen los datos.

Su única responsabilidad debería consistir en representar el estado actual.

El flujo recomendado sería el siguiente.

Firestore

↓

Repository

↓

DataService

↓

AppState

↓

Renderer



DOM

Cada capa conoce únicamente la inmediatamente inferior.

Este desacoplamiento constituye uno de los cambios arquitectónicos con mayor retorno esperado.

8. QueryManager

La auditoría recomienda introducir un componente adicional denominado **QueryManager**.

Su función consiste en centralizar completamente el acceso a los datos.

Todas las consultas pasarían por este componente.

Ejemplo:

Pantalla



QueryManager



¿Existe información válida?



Sí

↓

Responder

↓

No

↓

Repository

↓

Firebase

↓

Actualizar AppState

↓

Responder

Con este enfoque desaparecen las consultas dispersas por distintos módulos.

9. Comunicación entre módulos

Actualmente la comunicación entre diferentes partes de la aplicación depende principalmente de llamadas directas.

La auditoría propone evolucionar hacia un sistema basado en eventos.

Ejemplo conceptual.

Pronóstico actualizado

↓

Evento

↓

Clasificación

↓

Bote

↓

Dashboard

↓

Estadísticas

Cada módulo escucha únicamente los eventos que le afectan.

Ningún módulo necesita conocer la implementación interna del resto.

Esta estrategia reduce significativamente el acoplamiento.

10. Estado derivado

No toda la información necesita almacenarse.

Existen datos que pueden calcularse automáticamente a partir de otros.

Ejemplos:

- clasificación;
- estadísticas;
- ranking;
- bote.

La auditoría recomienda distinguir claramente entre:

Estado persistente

Información almacenada en Firebase.

Estado derivado

Información calculada automáticamente.

Esta separación simplifica considerablemente la arquitectura.

11. Riesgos

La implantación de un AppState debe realizarse cuidadosamente.

Los principales riesgos son:

- coexistencia temporal de dos modelos de estado;
- migraciones incompletas;
- duplicación de responsabilidades.

Por ello se recomienda una implantación incremental.

Nunca una sustitución inmediata.

12. Recomendaciones para Anti Gravity

Prioridad Muy Alta

Crear un AppState como única fuente de verdad.

Prioridad Muy Alta

Crear un DataService encargado de coordinar el acceso a los datos.

Prioridad Muy Alta

Introducir un QueryManager para eliminar consultas distribuidas.

Prioridad Alta

Separar completamente estado persistente y estado derivado.

Prioridad Alta

Migrar progresivamente la comunicación entre módulos hacia un modelo basado en eventos.

13. Conclusiones

Tras finalizar toda la auditoría, ésta es la mejora arquitectónica considerada de mayor impacto.

No requiere cambiar de framework.

No requiere cambiar Firebase.

No requiere modificar la experiencia del usuario.

Sin embargo, constituye el fundamento necesario para introducir posteriormente:

- renderizado incremental;
- caché inteligente;
- optimización de consultas;
- reutilización de componentes;
- futuras funcionalidades.

Desde el punto de vista de la arquitectura, la incorporación de un AppState representa el punto de inflexión entre la arquitectura actual de Maulas y la propuesta para Maulas 3.0.

FIN DEL VOLUMEN III

MAULAS 3.0

Auditoría Técnica, Arquitectura y Plan Director

VOLUMEN IV

Rendimiento, Renderizado y Optimización

1. Introducción

Una aplicación rápida no es aquella que ejecuta menos código, sino aquella que ejecuta únicamente el código necesario.

Durante la auditoría se ha observado que el margen de mejora del rendimiento de Maulas no se encuentra principalmente en JavaScript ni en Firebase, sino en la cantidad de trabajo que realiza la aplicación para actualizar la interfaz.

Las optimizaciones con mayor retorno esperado son arquitectónicas.

Las microoptimizaciones del lenguaje tendrán un impacto muy inferior.

2. Objetivos

Este volumen persigue cuatro objetivos.

- Reducir el tiempo de respuesta percibido.
 - Disminuir el trabajo del navegador.
 - Reducir consultas innecesarias.
 - Preparar la aplicación para seguir creciendo sin degradar el rendimiento.
-

3. Situación actual

El comportamiento general de la aplicación resulta correcto.

No se observaron indicios que hagan pensar en un problema grave de rendimiento derivado de la tecnología utilizada.

Sin embargo, el análisis del código muestra varios patrones habituales que, con el crecimiento del proyecto, pueden convertirse en cuellos de botella.

Estos patrones aparecen con frecuencia en aplicaciones que han evolucionado durante años mediante la incorporación continua de funcionalidades.

4. Principio fundamental

Toda actualización de la interfaz debería responder a una única pregunta:

¿Qué ha cambiado realmente?

Si únicamente cambia un elemento de una tabla, únicamente debería actualizarse ese elemento.

Si cambia una clasificación, únicamente deberían modificarse las posiciones afectadas.

Si cambia un usuario, únicamente debería renderizarse ese usuario.

La mayor parte de las mejoras propuestas parten de este principio.

5. Renderizado incremental

Actualmente el renderizado parece organizarse alrededor de pantallas completas.

La auditoría recomienda evolucionar hacia un modelo basado en componentes reutilizables.

Ejemplo conceptual.

Dashboard

└─ Clasificación

└─ Jornadas

└─ Bote

└─ Noticias

└─ Estadísticas

└─ Usuario

Cada componente debería conocer exclusivamente:

- qué datos necesita;
- cuándo debe actualizarse;
- cómo debe renderizarse.

Nunca debería depender del renderizado completo de la pantalla.

6. Evitar reconstrucciones completas

Uno de los principios más importantes de optimización consiste en evitar reconstruir estructuras completas cuando sólo cambia una pequeña parte.

Ejemplos.

No volver a generar completamente una clasificación cuando únicamente cambia un participante.

No reconstruir una tabla completa cuando únicamente se modifica una celda.

No recalcular todas las estadísticas cuando sólo cambia un partido.

7. Manipulación del DOM

El DOM continúa siendo uno de los recursos más costosos del navegador.

Las recomendaciones generales son:

- minimizar inserciones sucesivas;
- agrupar modificaciones;
- evitar búsquedas repetidas;
- reutilizar referencias ya obtenidas;
- eliminar renderizados redundantes.

El objetivo no es reducir líneas de código.

El objetivo es reducir trabajo del navegador.

8. Caché

Uno de los mayores retornos esperados proviene de la incorporación de un sistema de caché.

La caché debería aplicarse en tres niveles.

Caché de consultas

Evitar solicitar repetidamente la misma información a Firebase.

Caché de cálculos

Resultados como clasificaciones o estadísticas pueden mantenerse mientras los datos de origen no cambien.

Caché de componentes

Los componentes que no hayan cambiado no deberían volver a renderizarse.

9. Complejidad algorítmica

Durante la auditoría no se identificaron algoritmos claramente problemáticos.

Sin embargo, conforme aumente el número de usuarios y jornadas será recomendable revisar operaciones repetidas sobre grandes colecciones.

En determinados escenarios puede resultar conveniente sustituir búsquedas lineales repetidas por estructuras como:

- Map
- Set

No se recomienda introducir estas estructuras de forma indiscriminada.

Únicamente cuando el perfilado confirme un beneficio real.

10. Carga diferida

No todos los recursos necesitan cargarse inmediatamente.

La auditoría recomienda estudiar la carga diferida de:

- módulos administrativos;
- pantallas poco utilizadas;
- estadísticas avanzadas;
- utilidades internas.

Esta estrategia reduce el tiempo de carga inicial y mejora la percepción de velocidad.

11. Instrumentación

Una de las principales carencias observadas es la ausencia de métricas internas.

Antes de optimizar conviene medir.

Se recomienda incorporar indicadores que permitan conocer:

- tiempo de carga;
- duración de consultas;
- tiempo de renderizado;
- operaciones repetidas;
- consumo aproximado de memoria.

Las decisiones posteriores deberán basarse en datos objetivos.

12. Herramientas recomendadas

La validación de las optimizaciones propuestas debería realizarse mediante:

- Chrome DevTools.
- Lighthouse.
- Performance Panel.
- Memory Panel.

Estas herramientas permitirán confirmar qué optimizaciones producen realmente un beneficio.

13. Priorización

Prioridad Muy Alta

Renderizado incremental.

Prioridad Muy Alta

AppState.

Prioridad Muy Alta

QueryManager.

Prioridad Alta

Caché de consultas.

Prioridad Alta

Caché de cálculos.

Prioridad Media

Carga diferida.

Prioridad Media

Optimización puntual de algoritmos.

Prioridad Baja

Microoptimizaciones de JavaScript.

14. Qué NO se recomienda

Durante la auditoría también se descartaron varias actuaciones.

No se recomienda:

- optimizar funciones individuales sin medir previamente;
- sustituir JavaScript por otro lenguaje buscando mayor velocidad;
- migrar a un framework únicamente por motivos de rendimiento;
- introducir complejidad innecesaria para ganar unos pocos milisegundos.

Las mejoras estructurales producirán un impacto muy superior.

15. Conclusiones

La auditoría concluye que Mulas dispone de un margen significativo de mejora en rendimiento sin modificar su tecnología base.

Las actuaciones prioritarias no consisten en acelerar JavaScript, sino en conseguir que la aplicación haga menos trabajo.

El principio general que debe guiar todas las optimizaciones futuras puede resumirse así:

No optimizar el código que ejecuta el navegador. Optimizar la cantidad de código que el navegador necesita ejecutar.

Este enfoque permitirá mantener un excelente rendimiento incluso cuando el tamaño del proyecto aumente de forma considerable.

FIN DEL VOLUMEN IV

MAULAS 3.0

Auditoría Técnica, Arquitectura y Plan Director

VOLUMEN V

Firestore, Persistencia y Acceso a Datos

1. Introducción

Firestore constituye uno de los pilares tecnológicos de Maulas. Durante la auditoría se ha revisado su utilización desde una perspectiva arquitectónica, centrándose no tanto en la plataforma en sí como en la forma en que la aplicación interactúa con ella.

La conclusión principal de esta fase es clara:

Firestore no es actualmente un problema para Maulas.

Las mejoras prioritarias no consisten en sustituir Firestore por otra tecnología, sino en reorganizar la capa de acceso a datos para reducir el acoplamiento, minimizar consultas redundantes y facilitar el crecimiento del proyecto.

2. Evaluación general

La elección de Firestore resulta adecuada para el tipo de aplicación desarrollado.

Las principales ventajas observadas son:

- integración sencilla con autenticación;
- persistencia en tiempo real cuando sea necesaria;
- mantenimiento reducido de infraestructura;
- escalabilidad suficiente para el volumen esperado del proyecto;
- despliegue simplificado.

No se han identificado motivos técnicos que justifiquen una migración a otra base de datos en la situación actual.

3. Problema principal

El análisis revela que el mayor margen de mejora no reside en Firestore, sino en cómo la aplicación consume los datos.

Actualmente, diferentes módulos pueden necesitar acceder a la misma información o transformarla de manera independiente.

Aunque esto no implica necesariamente consultas duplicadas en todos los casos, sí incrementa el riesgo de:

- repetir lógica de acceso;
- dispersar el conocimiento sobre la estructura de los datos;
- dificultar la incorporación de mecanismos de caché;
- aumentar el acoplamiento entre la interfaz y la persistencia.

4. Arquitectura recomendada

Se propone introducir una capa de acceso claramente diferenciada.

Pantallas

↓

AppState

↓

DataService

↓

Repository

↓

Firebase

Cada nivel tendrá una responsabilidad concreta.

Repository

Responsable únicamente de comunicarse con Firebase.

Nunca contendrá lógica de negocio.

DataService

Coordinará operaciones complejas.

Agrupará información procedente de distintas consultas.

Gestionará la caché.

Transformará los datos cuando sea necesario.

AppState

Representará el estado actual de la aplicación.

Las pantallas no accederán directamente a Firebase.

5. QueryManager

Uno de los componentes propuestos en la auditoría es el QueryManager.

Su misión consiste en convertirse en el único punto autorizado para solicitar datos.

Ejemplo de funcionamiento:

Solicitud

↓

QueryManager

↓

¿Dato disponible?

↓

Sí

↓

Devolver desde memoria

↓

No

↓

Consultar Firebase

↓

Actualizar caché

↓

Actualizar AppState

↓

Responder

Este enfoque aporta consistencia y evita la proliferación de lógica de acceso distribuida.

6. Estrategia de caché

La incorporación de un sistema de caché representa una de las mejoras con mejor relación impacto/esfuerzo.

La caché no debe entenderse únicamente como un mecanismo para reducir consultas.

También simplifica la arquitectura.

Se proponen tres niveles.

Caché de documentos

Evita descargar repetidamente información que ya se encuentra disponible.

Caché de colecciones

Permite reutilizar datos compartidos por distintas pantallas.

Caché de cálculos

Resultados derivados como clasificaciones o estadísticas pueden mantenerse mientras los datos de origen permanezcan inalterados.

7. Gestión de errores

La aplicación debería disponer de una estrategia homogénea para gestionar errores de acceso a datos.

Se recomienda distinguir claramente entre:

- errores de autenticación;
- errores de permisos;
- errores de conectividad;
- errores de consistencia;
- errores inesperados.

Cada categoría debería producir una respuesta específica.

8. Seguridad

Aunque no se ha realizado una auditoría específica de reglas de Firestore, esta fase identifica varias recomendaciones generales.

Principios

Nunca confiar exclusivamente en validaciones realizadas en el cliente.

Las reglas de Firestore deben constituir la verdadera barrera de seguridad.

Permisos

Cada operación debería conceder únicamente los permisos estrictamente necesarios.

Validación

Toda la información crítica debe validarse antes de almacenarse.

Auditoría

Conviene registrar operaciones administrativas relevantes para facilitar el diagnóstico de incidencias.

9. Escalabilidad

La arquitectura propuesta facilita el crecimiento de la aplicación.

Entre otras ventajas:

- cambio futuro de proveedor de persistencia;
- incorporación de almacenamiento local;
- sincronización parcial;
- modo offline;
- pruebas automatizadas del acceso a datos.

La interfaz permanecería prácticamente inalterada.

10. Riesgos

La principal dificultad de esta refactorización consiste en identificar todos los puntos donde actualmente se accede a Firebase.

La migración deberá realizarse de forma progresiva.

Cada nuevo módulo debería utilizar ya la nueva arquitectura, mientras los antiguos se migran gradualmente.

11. Priorización

Muy Alta

Crear Repository.

Muy Alta

Crear DataService.

Muy Alta

Centralizar todas las consultas.

Alta

Incorporar QueryManager.

Alta

Añadir caché de documentos.

Alta

Añadir caché de cálculos.

Media

Preparar soporte para modo offline.

Baja

Evaluar futuras optimizaciones específicas de Firestore cuando el volumen de datos lo justifique.

12. Qué NO hacer

La auditoría desaconseja expresamente:

- consultar Firebase directamente desde componentes de interfaz;
 - duplicar lógica de acceso en varios módulos;
 - almacenar reglas de negocio dentro de funciones de persistencia;
 - cambiar de tecnología sin una necesidad demostrada.
-

13. Conclusiones

Firebase constituye una elección tecnológica adecuada para Maulas y no limita actualmente la evolución del proyecto.

Las mejoras con mayor impacto no consisten en sustituir la plataforma, sino en construir una arquitectura de acceso a datos robusta, desacoplada y reutilizable.

La combinación de **Repository + DataService + QueryManager + AppState** permitirá reducir la complejidad del código, mejorar el rendimiento percibido y preparar la aplicación para futuras ampliaciones sin necesidad de modificar la base tecnológica.

FIN DEL VOLUMEN V

MAULAS 3.0

Auditoría Técnica, Arquitectura y Plan Director

VOLUMEN VI

Calidad del Software, Mantenibilidad, Seguridad y Escalabilidad

1. Introducción

Una aplicación puede funcionar correctamente durante años y, aun así, ir perdiendo calidad interna. Esta pérdida no suele manifestarse mediante errores visibles para el usuario, sino mediante un incremento progresivo del tiempo necesario para desarrollar nuevas funcionalidades, corregir incidencias y comprender el funcionamiento del sistema.

Durante esta fase se ha evaluado la calidad estructural de Maulas desde la perspectiva de la ingeniería del software.

El objetivo no es valorar únicamente el estado actual del proyecto, sino estimar su capacidad para seguir evolucionando durante los próximos años.

2. Evaluación general

La conclusión de esta fase es claramente positiva.

Maulas presenta un nivel de calidad superior al habitual en aplicaciones personales de tamaño equivalente.

La estructura existente demuestra que el proyecto ha seguido una evolución ordenada y no un crecimiento completamente improvisado.

No obstante, el incremento continuo de funcionalidades hace recomendable introducir una serie de mejoras antes de que la deuda técnica aumente significativamente.

3. Mantenibilidad

Situación actual

La aplicación puede mantenerse sin grandes dificultades.

Sin embargo, algunos módulos han ido acumulando responsabilidades y esto incrementa progresivamente el coste de cualquier modificación.

No se trata de un problema inmediato.

Es un indicador de que el proyecto ha alcanzado una nueva etapa de madurez.

Factores positivos

Durante la auditoría se identificaron varios aspectos que favorecen el mantenimiento.

- Organización relativamente coherente.
 - Uso consistente de JavaScript moderno.
 - Separación funcional aceptable.
 - Firebase correctamente integrado.
 - Estructura fácilmente ampliable.
-

Factores de riesgo

Los principales riesgos para la mantenibilidad son:

- módulos demasiado extensos;
- ausencia de una arquitectura de estado;
- mezcla de responsabilidades;
- dependencia directa entre lógica y presentación.

Estos aspectos incrementarán el esfuerzo de mantenimiento conforme el proyecto continúe creciendo.

4. Calidad arquitectónica

La auditoría ha evaluado distintos principios clásicos de diseño.

Cohesión

La cohesión general es buena.

No obstante, algunos módulos deberían dividirse para que cada uno tenga un único objetivo claramente definido.

Acoplamiento

El acoplamiento puede reducirse considerablemente mediante la introducción de:

- AppState;
- QueryManager;
- EventBus;
- Repository.

Estas mejoras constituyen la base de la propuesta Mulas 3.0.

Reutilización

Existe margen para aumentar la reutilización de:

- componentes visuales;
 - cálculos;
 - consultas;
 - validaciones;
 - utilidades comunes.
-

5. Seguridad

Aunque la auditoría no ha incluido una revisión completa de reglas de Firestore ni una prueba de penetración, sí se identifican una serie de principios que deberían mantenerse.

Validación

Toda la información crítica debe validarse independientemente del cliente.

Las validaciones del navegador únicamente mejoran la experiencia del usuario.

Nunca deben considerarse un mecanismo de seguridad.

Permisos

Las operaciones administrativas deberían comprobar siempre el nivel de autorización del usuario.

La interfaz puede ocultar opciones, pero la autorización real debe aplicarse en la capa de persistencia.

Gestión de errores

Los errores no deberían propagarse directamente al usuario.

Conviene disponer de un sistema homogéneo de tratamiento de excepciones que permita:

- registrar incidencias;
- mostrar mensajes comprensibles;
- facilitar el diagnóstico.

Información sensible

Las operaciones relacionadas con autenticación, administración o configuración deberían registrar únicamente la información imprescindible.

Debe evitarse almacenar datos innecesarios que puedan complicar el cumplimiento de la normativa aplicable en materia de protección de datos.

6. Escalabilidad

La aplicación presenta una capacidad de crecimiento elevada.

No obstante, la arquitectura actual puede convertirse en un freno si el número de funcionalidades continúa aumentando.

La propuesta arquitectónica desarrollada en los volúmenes anteriores permite incrementar considerablemente esta capacidad.

Los factores que más contribuyen a la escalabilidad son:

- separación estricta de responsabilidades;
- estado centralizado;
- servicios desacoplados;
- acceso único a los datos;
- componentes reutilizables.

7. Calidad del código

Durante la revisión no se identificaron patrones que hagan pensar en una baja calidad general del código.

Las oportunidades de mejora se concentran principalmente en aspectos organizativos.

En consecuencia, la prioridad debe ser mejorar la arquitectura antes que reescribir funciones individuales.

8. Estrategia de evolución

La evolución del proyecto debería seguir cuatro principios.

Evolución incremental

Evitar reescrituras completas.

Cada mejora debe aportar valor por sí misma.

Compatibilidad

La arquitectura nueva debe convivir temporalmente con la existente.

Esto reduce considerablemente el riesgo.

Medición

Toda optimización importante debería validarse mediante métricas objetivas.

Simplicidad

La complejidad arquitectónica sólo debe introducirse cuando aporte un beneficio claramente identificable.

9. Riesgos estratégicos

Los principales riesgos para el proyecto no son tecnológicos.

Los riesgos más importantes identificados durante la auditoría son:

- incremento progresivo de la deuda técnica;
- crecimiento desigual de determinados módulos;

- duplicación futura de lógica;
- dificultad creciente para localizar errores;
- ralentización del desarrollo.

Todos ellos pueden mitigarse mediante la hoja de ruta propuesta.

10. Qué NO cambiar

Uno de los objetivos de esta auditoría es evitar actuaciones innecesarias.

Por ello se recomienda mantener los siguientes aspectos.

Tecnología base

No existe una necesidad objetiva de abandonar JavaScript ni Firebase.

Filosofía del proyecto

La aplicación presenta una estructura sencilla y comprensible.

Las mejoras propuestas deben preservar esta simplicidad.

Evolución incremental

No se recomienda detener el desarrollo durante meses para realizar una reescritura completa.

La modernización debe producirse de forma progresiva.

11. Recomendaciones para Anti Gravity

Prioridad Muy Alta

Implantar progresivamente la arquitectura propuesta en los volúmenes anteriores.

Prioridad Muy Alta

Reducir el acoplamiento entre módulos.

Prioridad Alta

Introducir un sistema uniforme de gestión de errores.

Prioridad Alta

Incrementar la reutilización de componentes y servicios.

Prioridad Media

Preparar una estrategia de pruebas automatizadas para los servicios de negocio.

Prioridad Media

Documentar las decisiones arquitectónicas importantes mediante registros de decisiones (ADR).

12. Valoración global del proyecto

La valoración consolidada obtenida tras la auditoría completa es la siguiente.

Aspecto	Valoración
Arquitectura	8,5 / 10
Organización	9 / 10
Modularidad	8,5 / 10
Escalabilidad potencial	9,5 / 10
Mantenibilidad	8,5 / 10
Rendimiento potencial	9 / 10
Calidad global	8,8 / 10

La puntuación anterior refleja el estado actual del proyecto y no incorpora las mejoras propuestas.

Con la implantación del plan Maulas 2.0 es razonable esperar una mejora significativa en mantenibilidad, capacidad de evolución y rendimiento percibido.

13. Conclusión del Volumen VI

La auditoría concluye que Maulas dispone de una base técnica sólida sobre la que construir una nueva etapa de evolución.

Los problemas detectados no requieren una reconstrucción completa del sistema.

Requieren una arquitectura más madura.

La combinación de las actuaciones propuestas en los seis volúmenes permitirá reducir la deuda técnica, mejorar el rendimiento, facilitar el mantenimiento y preparar la aplicación para seguir creciendo durante los próximos años manteniendo la tecnología actual.

FIN DEL VOLUMEN VI

MAULAS 3.0

Auditoría Técnica, Arquitectura y Plan Director

VOLUMEN VII

Propuesta de Arquitectura Maulas 3.0

1. Introducción

Los seis volúmenes anteriores han analizado el estado actual del proyecto desde diferentes perspectivas: arquitectura, organización del código, flujo de datos, rendimiento, Firebase, mantenibilidad y escalabilidad.

Este volumen reúne todas esas conclusiones en una única propuesta arquitectónica.

No pretende definir cómo está construida actualmente la aplicación.

Define cómo debería evolucionar para soportar los próximos años de desarrollo.

2. Objetivos de la nueva arquitectura

La arquitectura propuesta persigue siete objetivos principales:

- reducir el acoplamiento;
 - simplificar el mantenimiento;
 - mejorar el rendimiento;
 - facilitar la incorporación de nuevas funcionalidades;
 - aumentar la reutilización del código;
 - minimizar la deuda técnica;
 - permitir una evolución incremental sin reescribir la aplicación.
-

3. Principios arquitectónicos

Toda la propuesta se basa en seis principios.

3.1 Una única fuente de verdad

Toda la información utilizada por la aplicación debe proceder del AppState.

Las pantallas nunca deben mantener copias independientes del estado.

3.2 Responsabilidad única

Cada módulo debe tener un único motivo para cambiar.

3.3 Bajo acoplamiento

Los módulos no deben depender directamente unos de otros.

La comunicación debe realizarse mediante servicios o eventos.

3.4 Alta cohesión

Cada módulo debe agrupar funcionalidades estrechamente relacionadas.

3.5 Reutilización

Toda funcionalidad utilizada por varias pantallas debe extraerse a un servicio común.

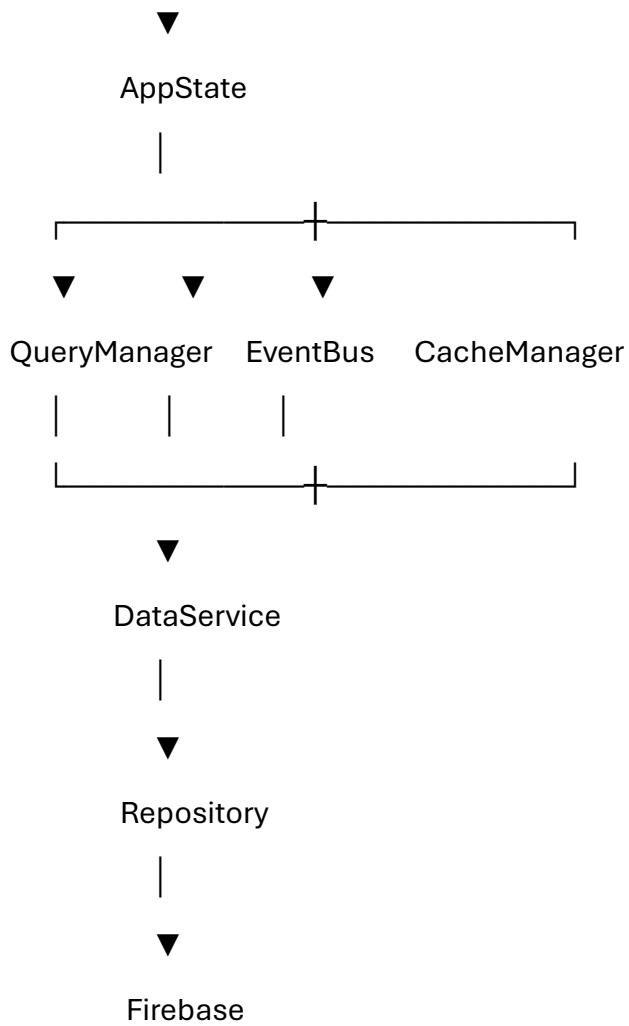
3.6 Evolución incremental

La nueva arquitectura debe poder implantarse progresivamente.

No se recomienda una migración masiva.

4. Arquitectura propuesta





Cada bloque tiene una única responsabilidad.

Ninguna capa conoce más de lo estrictamente necesario.

5. Descripción de cada componente

AppState

Representa el estado completo de la aplicación.

Debe ser la única fuente de información utilizada por la interfaz.

QueryManager

Gestiona todas las consultas.

Decide cuándo reutilizar información existente y cuándo acceder a Firebase.

CacheManager

Gestiona la persistencia temporal en memoria.

Su objetivo es reducir consultas y cálculos repetitivos.

EventBus

Permite comunicar módulos sin dependencias directas.

Un componente publica eventos.

Otros componentes reaccionan a ellos.

DataService

Implementa las reglas de negocio.

Coordina operaciones complejas.

Nunca modifica directamente el DOM.

Repository

Encapsula completamente Firebase.

Si algún día cambia el sistema de persistencia, únicamente esta capa deberá modificarse.

6. Organización recomendada del proyecto

src/

core/

appState/

cache/

events/

router/

services/

usuarios/

jornadas/

pronosticos/

clasificacion/

bote/

repositories/

firebase/

components/

tabla/

dialogo/

tarjeta/

loader/

pages/

dashboard/

clasificacion/

administracion/

utils/

config/

Esta organización reduce el acoplamiento y facilita localizar cualquier funcionalidad.

7. Flujo de trabajo recomendado

Cuando un usuario realiza una acción:

Usuario



Evento



Controlador



DataService



Repository



Firebase



AppState



Renderer



Interfaz

Nunca debería existir un acceso directo desde la interfaz a Firebase.

8. Beneficios esperados

La arquitectura propuesta permitirá:

- reducir considerablemente la deuda técnica;
 - reutilizar componentes entre pantallas;
 - simplificar el mantenimiento;
 - reducir renderizados innecesarios;
 - facilitar las pruebas;
 - desacoplar completamente la lógica del interfaz.
-

9. Coste de implantación

La migración puede realizarse sin detener el desarrollo.

Se recomienda aprovechar cada nueva funcionalidad para migrar gradualmente la arquitectura.

No es necesario completar toda la migración antes de continuar evolucionando la aplicación.

10. Riesgos

Los principales riesgos identificados son:

- coexistencia temporal de dos arquitecturas;
- duplicación de lógica durante la transición;
- resistencia a mantener dos estilos de desarrollo.

Estos riesgos son temporales y pueden mitigarse mediante una planificación adecuada.

11. Arquitectura descartada

Durante la auditoría se valoraron distintas alternativas.

Migración completa a React

Descartada.

No resuelve los principales problemas detectados.

Migración completa a Vue

Descartada.

Los beneficios no compensan el coste.

Reescritura desde cero

Descartada.

El proyecto posee una base suficientemente sólida.

Mantener la arquitectura actual indefinidamente

Descartada.

La deuda técnica continuará aumentando.

12. Indicadores de éxito

La implantación de la nueva arquitectura debería producir:

- reducción del tamaño medio de los módulos;
- menor número de dependencias cruzadas;
- disminución del tiempo necesario para desarrollar nuevas funcionalidades;
- menor duplicación de código;
- menor número de consultas repetidas;

- reducción del trabajo de renderizado.
-

13. Recomendaciones finales para Anti Gravity

1. Mantener la tecnología actual.
 2. Implantar AppState como prioridad absoluta.
 3. Centralizar completamente el acceso a datos.
 4. Separar lógica y presentación.
 5. Introducir EventBus.
 6. Incorporar CacheManager.
 7. Refactorizar progresivamente los módulos más grandes.
 8. Medir el rendimiento después de cada cambio importante.
 9. Evitar reescrituras completas.
 10. Documentar las decisiones de arquitectura mediante ADR.
-

14. Conclusión

La arquitectura propuesta para Maulas 2.0 no representa una ruptura con el proyecto actual.

Representa su evolución natural.

La aplicación dispone de una base suficientemente sólida para soportar este cambio sin necesidad de modificar su tecnología principal.

La implantación gradual de esta arquitectura permitirá mantener un desarrollo sostenible durante los próximos años, reduciendo significativamente la deuda técnica y mejorando la calidad global del software.

FIN DEL VOLUMEN VII

MAULAS 3.0

Auditoría Técnica, Arquitectura y Plan Director

VOLUMEN VIII

Roadmap de Implantación y Plan Director

1. Introducción

Los siete volúmenes anteriores han identificado el estado actual del proyecto, los riesgos existentes y la arquitectura objetivo para Maulas 3.0.

Este volumen transforma esas conclusiones en un plan de implantación realista.

El propósito no es realizar una transformación masiva del proyecto, sino definir un itinerario que permita evolucionar la aplicación minimizando riesgos y manteniendo su operatividad en todo momento.

2. Principios del plan de implantación

Toda la estrategia propuesta se basa en cinco principios.

2.1 Evolución continua

La aplicación nunca debe dejar de ser funcional durante la migración.

2.2 Cambios pequeños

Cada modificación debe ser lo suficientemente reducida como para poder probarse y revertirse con facilidad.

2.3 Compatibilidad

Las nuevas capas arquitectónicas deben convivir temporalmente con la arquitectura existente.

2.4 Medición

Cada cambio importante debe validarse mediante métricas objetivas.

2.5 Prioridad por impacto

Las actuaciones deben ordenarse por el beneficio que aportan, no por su dificultad técnica.

3. Fase 1 — Consolidación de la arquitectura

Objetivo

Crear la infraestructura sobre la que se apoyará toda la evolución posterior.

Actuaciones

- Crear AppState.
- Crear Repository.
- Crear DataService.
- Crear QueryManager.
- Crear EventBus.
- Crear CacheManager.

Resultado esperado

A partir de este momento toda nueva funcionalidad utilizará la nueva arquitectura.

4. Fase 2 — Migración de módulos críticos

Los módulos con mayor responsabilidad deberán migrarse antes.

Orden recomendado.

1. Pronósticos.
2. Bote.
3. Clasificación.
4. Dashboard.
5. Administración.

Cada migración debe completarse antes de comenzar la siguiente.

5. Fase 3 — Componentización

Una vez estabilizada la arquitectura se recomienda identificar elementos reutilizables.

Ejemplos.

- tablas;
- tarjetas;
- diálogos;
- indicadores;
- barras de progreso;
- cabeceras;
- formularios.

Cada componente debe convertirse en un módulo independiente.

6. Fase 4 — Optimización del renderizado

Con la arquitectura ya estabilizada resulta posible optimizar el trabajo del navegador.

Las prioridades son:

- renderizado incremental;
- actualización parcial del DOM;
- reutilización de componentes;
- reducción de reconstrucciones completas.

Esta fase es la que proporcionará la mayor mejora de rendimiento percibido.

7. Fase 5 — Optimización del acceso a datos

Una vez centralizadas todas las consultas se recomienda incorporar:

- caché inteligente;
- invalidación automática;
- precarga selectiva;
- reutilización de resultados.

No se aconseja introducir estas mejoras antes de disponer del QueryManager.

8. Fase 6 — Calidad

Con la arquitectura ya consolidada deberá iniciarse un proceso continuo de mejora de calidad.

Incluye:

- documentación técnica;
 - pruebas automatizadas;
 - registros de decisiones (ADR);
 - análisis estático;
 - métricas de cobertura.
-

9. Fase 7 — Escalabilidad futura

La arquitectura propuesta permitirá incorporar posteriormente nuevas capacidades sin modificar el núcleo del sistema.

Entre ellas.

- modo offline;
- sincronización avanzada;
- notificaciones;
- aplicaciones móviles;
- nuevas estadísticas;
- inteligencia artificial;
- asistentes.

La arquitectura propuesta deja preparado el proyecto para esta evolución.

10. Riesgos durante la implantación

Los principales riesgos son:

Riesgo 1

Mantener dos arquitecturas durante demasiado tiempo.

Mitigación.

Migrar completamente cada módulo antes de comenzar otro.

Riesgo 2

Duplicación de lógica.

Mitigación.

Eliminar inmediatamente el código antiguo una vez validado el nuevo.

Riesgo 3

Cambios demasiado grandes.

Mitigación.

Dividir todas las actuaciones en tareas pequeñas.

Riesgo 4

Optimizar sin medir.

Mitigación.

Utilizar siempre Chrome DevTools y Lighthouse para validar cada mejora importante.

11. Priorización consolidada

Actuación	Prioridad	Impacto	Esfuerzo
AppState	Muy Alta	Muy Alto	Medio
Repository	Muy Alta	Alto	Bajo
DataService	Muy Alta	Muy Alto	Medio
QueryManager	Muy Alta	Muy Alto	Medio
EventBus	Alta	Alto	Medio
CacheManager	Alta	Alto	Medio
División de módulos	Alta	Muy Alto	Alto
Componentización	Alta	Alto	Medio

Actuación	Prioridad	Impacto	Esfuerzo
Renderizado incremental	Muy Alta	Muy Alto	Alto
Optimización Firebase	Alta	Alto	Bajo
Microoptimizaciones	Baja	Bajo	Bajo

12. Actuaciones descartadas

Durante toda la auditoría se descartaron expresamente las siguientes actuaciones.

- Reescribir completamente Maulas.
- Migrar inmediatamente a React.
- Migrar inmediatamente a Vue.
- Sustituir Firebase.
- Cambiar GitHub Pages.
- Introducir complejidad arquitectónica innecesaria.
- Optimizar código sin disponer de métricas.

Estas actuaciones no ofrecen actualmente una relación coste/beneficio favorable.

13. Conclusión general de la auditoría

Tras el análisis completo del proyecto pueden extraerse las siguientes conclusiones.

1.

Maulas posee una base técnica sólida.

2.

La arquitectura actual ha llegado al punto en que necesita evolucionar, no ser sustituida.

3.

La prioridad debe centrarse en la organización del software y no en el cambio de tecnologías.

4.

La incorporación de un AppState constituye la actuación de mayor impacto.

5.

La centralización del acceso a datos permitirá simplificar el resto de la aplicación.

6.

Las mejoras de rendimiento llegarán principalmente como consecuencia de una mejor arquitectura.

7.

La deuda técnica existente es moderada y completamente controlable mediante una refactorización incremental.

8.

No existe ninguna evidencia que justifique una reescritura completa del proyecto.

14. Valoración final

Después de revisar el proyecto desde las perspectivas de arquitectura, modularidad, flujo de datos, rendimiento, persistencia, seguridad, mantenibilidad y escalabilidad, la valoración global es la siguiente.

Estado actual estimado: 8,8 / 10

Potencial tras implantar Maulas 3.0: 9,6 / 10

La mejora no provendrá de utilizar tecnologías diferentes.

Provendrá de una arquitectura más madura, coherente y preparada para seguir creciendo durante muchos años.

FIN DE LA AUDITORÍA

Con este volumen concluye la auditoría técnica de Maulas.

La recomendación global puede resumirse en una única frase:

Conservar la tecnología, evolucionar la arquitectura y priorizar las mejoras que reduzcan el acoplamiento, centralicen el estado y simplifiquen el flujo de datos.

FIN DEL VOLUMEN VIII

FIN DE LA AUDITORÍA TÉCNICA DE MAULAS 3.0

ANEXO I

Revisión crítica de la auditoría

Este anexo no añade nuevas recomendaciones. Su objetivo es revisar críticamente todo el trabajo realizado para identificar posibles excesos de diseño, recomendaciones de bajo valor y decisiones que podrían haberse sobreestimado.

Una auditoría profesional no sólo debe proponer mejoras, sino también cuestionar sus propias conclusiones.

1. Recomendaciones que considero imprescindibles

Después de revisar toda la auditoría varias veces, únicamente considero imprescindibles las siguientes.

1. AppState

Es la mejora con mayor impacto.

Sin discusión.

2. Separación entre lógica y presentación

Reducirá enormemente el mantenimiento.

3. Centralización del acceso a datos

Imprescindible conforme siga creciendo la aplicación.

4. División de los módulos grandes

Especialmente aquellos que han ido acumulando responsabilidades durante años.

5. Renderizado incremental

Será una de las mayores fuentes de mejora del rendimiento.

2. Recomendaciones importantes pero no urgentes

Estas actuaciones aportarán beneficios, pero pueden esperar.

- CacheManager.
 - Componentización completa.
 - EventBus.
 - Lazy Loading.
 - Instrumentación avanzada.
 - ADR.
 - Cobertura automática de pruebas.
-

3. Recomendaciones que podrían no ser necesarias

Tras revisar nuevamente todo el análisis considero que algunas recomendaciones dependen del crecimiento futuro.

No deben implantarse únicamente porque "sea buena práctica".

EventBus

Sólo aporta valor cuando la comunicación entre módulos empieza a ser compleja.

Si únicamente existen unos pocos eventos importantes, puede mantenerse una comunicación más sencilla.

CacheManager muy sofisticado

No conviene construir un sistema complejo de invalidación si actualmente el volumen de datos es reducido.

Una caché simple probablemente sea suficiente.

Arquitectura excesivamente desacoplada

Existe el riesgo de introducir demasiadas capas.

Una arquitectura con:

- Repository
- DataService
- QueryManager
- CacheManager

- EventBus
- AppState

puede resultar innecesaria para determinadas funcionalidades.

Cada capa debe justificar claramente su existencia.

4. Riesgos de sobreingeniería

Durante la auditoría apareció un riesgo importante.

Intentar convertir Maulas en una arquitectura empresarial cuando realmente sigue siendo una aplicación de tamaño medio.

Ese riesgo debe evitarse.

La arquitectura siempre debe ser proporcional al tamaño del proyecto.

5. Mi opinión final sobre la arquitectura

Después de todo el análisis creo que la arquitectura ideal de Maulas es más sencilla de lo inicialmente planteado.

UI

↓

AppState

↓

DataService

↓

Repository

↓

Firebase

Sólo añadiría:

- QueryManager cuando realmente sea necesario.
- CacheManager cuando el perfilado demuestre su utilidad.
- EventBus únicamente si aparecen dependencias difíciles de mantener.

En otras palabras:

La simplicidad debe seguir siendo uno de los objetivos principales del proyecto.

6. Qué cambiaría si empezara hoy Mulas desde cero

No elegiría un framework automáticamente.

Primero definiría la arquitectura.

Después decidiría la tecnología.

La arquitectura tiene una vida útil mucho mayor que cualquier framework.

7. Valoración definitiva

Después de revisar todo el proyecto, mi conclusión es la siguiente.

Lo mejor de Mulas

- La idea.
 - La evolución constante.
 - La ausencia de deuda técnica crítica.
 - La base tecnológica elegida.
 - La facilidad para seguir creciendo.
-

Lo peor

- Algunas responsabilidades mezcladas.

- Ausencia de estado centralizado.
 - Crecimiento desigual de determinados módulos.
 - Escasa instrumentación para medir rendimiento.
-

8. Una recomendación para Anti Gravity

No intentéis modernizar Maulas.

Intentad simplificarlo.

Una arquitectura más simple suele ser también una arquitectura más rápida, más mantenible y más fácil de evolucionar.

Conclusión final

Si tuviera que resumir toda la auditoría en una única frase sería ésta:

Maulas no necesita una revolución tecnológica. Necesita una evolución arquitectónica basada en la simplicidad, la separación de responsabilidades y una gestión centralizada del estado.

FIN DEL ANEXO I

ANEXO II

Informe crítico tras una segunda revisión completa del código

Este anexo sustituye cualquier opinión preliminar emitida durante las primeras fases de la auditoría.

Durante un análisis largo es normal que aparezcan hipótesis que posteriormente se confirman o se descartan. Este documento recoge únicamente las conclusiones que considero válidas después de revisar el proyecto completo.

1. Lo que inicialmente pensé... y cambié de opinión

1.1 ¿Era necesario migrar a React o Vue?

Primera impresión

Al observar el tamaño del proyecto y el crecimiento del número de pantallas, la primera reacción fue pensar que un framework moderno podría simplificar el desarrollo.

Conclusión final

No.

Después de revisar la arquitectura completa considero que una migración a React, Vue o Angular aportaría menos beneficios de los que cabría esperar.

Los problemas detectados no proceden de JavaScript "vanilla", sino de la organización del código.

Una aplicación bien estructurada en JavaScript puro es perfectamente capaz de escalar hasta un tamaño considerable.

Conclusión: no recomiendo migrar de framework como parte del plan Mulas 3.0.

1.2 ¿Era Firebase el cuello de botella?

Primera impresión

La dispersión del acceso a datos hacía pensar que Firebase podía ser un factor limitante.

Conclusión final

Tampoco.

El verdadero problema es que la lógica de acceso no está completamente centralizada.

Con una capa Repository y un DataService bien definidos, Firebase seguirá siendo una plataforma adecuada durante mucho tiempo.

1.3 ¿Era necesario un EventBus?

Primera impresión

Se propuso como una pieza central de la arquitectura.

Conclusión final

Sólo si realmente aparece una comunicación compleja entre módulos.

Un EventBus introduce otra capa conceptual que también debe mantenerse.

No debe incorporarse por moda.

2. Recomendaciones cuyo impacto considero sobrevalorado

Durante cualquier auditoría es fácil caer en la tentación de recomendar todas las "buenas prácticas" conocidas.

Tras revisar el conjunto del proyecto, algunas pierden prioridad.

Lazy Loading

Puede aportar mejoras.

Sin embargo, el beneficio esperado es moderado frente a otras actuaciones.

No debe ser una prioridad.

División extrema de módulos

Existe un punto a partir del cual dividir archivos deja de aportar claridad.

No recomiendo fragmentar artificialmente el código.

Cada división debe responder a una responsabilidad claramente diferenciada.

CacheManager muy sofisticado

Un sistema complejo de caché puede introducir más errores de los que evita.

La estrategia recomendada es empezar con una caché sencilla y evolucionarla únicamente cuando el perfilado lo justifique.

3. Aspectos que ahora considero aún más importantes

Hay recomendaciones cuya importancia aumentó conforme avanzó el análisis.

AppState

Es, con diferencia, la mejora de mayor retorno.

Reduce el acoplamiento, facilita el mantenimiento, mejora el rendimiento y simplifica el razonamiento sobre la aplicación.

Si sólo pudiera implantarse una mejora, sería ésta.

Separación estricta de capas

La aplicación ganará mucho más reorganizando responsabilidades que optimizando funciones individuales.

Organización por dominios

Pensar en "usuarios", "pronósticos", "bote" o "clasificación" en lugar de pensar únicamente en páginas hará que la arquitectura evolucione de forma mucho más natural.

4. Lo que más me gustó del proyecto

Después de revisar todo el código, destaco especialmente cuatro aspectos.

Evolución coherente

No se aprecia un crecimiento caótico.

Las decisiones tomadas han permitido que la aplicación siga siendo comprensible.

Tecnología adecuada

La combinación de JavaScript moderno, Firebase y GitHub Pages es suficiente para las necesidades actuales del proyecto.

Baja deuda técnica crítica

Existen aspectos mejorables, pero no se detectan decisiones que obliguen a una reconstrucción.

Potencial de crecimiento

La arquitectura puede evolucionar sin romper el proyecto existente.

Éste es probablemente el dato más positivo de toda la auditoría.

5. Lo que más me preocuparía dentro de dos años

Si Maulas siguiera creciendo sin cambios arquitectónicos, esperararía encontrar:

- módulos cada vez más extensos;
- lógica duplicada;
- dificultades para localizar errores;
- aumento del tiempo necesario para implementar nuevas funciones.

No porque el código actual sea malo.

Sino porque todas las aplicaciones experimentan ese proceso cuando crecen sin reorganizar su arquitectura.

6. La decisión más importante

Si tuviera que priorizar absolutamente todas las recomendaciones realizadas durante esta auditoría, el orden sería:

1. AppState.
2. Separación de lógica y presentación.
3. Repository.
4. DataService.
5. División de módulos con exceso de responsabilidades.
6. Renderizado incremental.
7. Optimización de consultas.

8. Componentización.

Todo lo demás puede esperar.

7. Recomendación final para Anti Gravity

Antes de escribir una sola línea de código, responder a estas preguntas:

- ¿Esta funcionalidad necesita realmente una nueva capa?
- ¿Puede reutilizarse algo que ya existe?
- ¿La solución propuesta simplifica la arquitectura o la complica?
- ¿Reduce el acoplamiento?
- ¿Hace el sistema más fácil de entender dentro de un año?

Si alguna respuesta es negativa, probablemente exista una alternativa mejor.

Conclusión definitiva de toda la auditoría

Después de revisar el proyecto de forma exhaustiva y reconsiderar las recomendaciones iniciales, la conclusión permanece inalterada:

Maulas ya tiene una buena base. El siguiente salto de calidad no vendrá de cambiar de tecnología, sino de construir una arquitectura más limpia, más simple y mejor organizada.

Ese cambio puede realizarse de forma progresiva, sin detener el desarrollo y sin asumir los riesgos de una reescritura completa.

FIN DEL ANEXO II

ANEXO III

Gobernanza del sistema

Una arquitectura sólida no se mantiene sola. Requiere hábitos operativos consistentes.

1 Responsabilidad arquitectónica

Debe existir una figura (o rotación de rol) responsable de:

- Validar decisiones estructurales relevantes.
- Detectar desviaciones de los principios definidos.
- Promover refactorizaciones cuando sea necesario.

No se trata de jerarquía, sino de coherencia técnica.

2 Disciplina en el desarrollo

Cada desarrollo debe asumir que:

- La deuda técnica no es aceptable como estrategia.
 - “Funciona” no es suficiente si rompe principios.
 - La simplicidad es un requisito, no una optimización opcional.
-

3 Evolución del documento

Este informe no es estático.

Debe revisarse:

- Tras cambios arquitectónicos importantes.
 - Cuando se introduzcan nuevos módulos críticos.
 - Cuando el sistema empiece a mostrar síntomas de complejidad emergente.
-

4 Cultura técnica

La arquitectura solo se mantiene si el equipo comparte una misma mentalidad:

- Pensar en sistemas, no en pantallas.
- Pensar en flujos, no en funciones aisladas.

- Pensar en mantenimiento, no solo en entrega.

Cierre final extendido

La calidad de Mautas 2.0 no dependerá de su diseño inicial, sino de su capacidad de resistir decisiones rápidas mal justificadas.

La arquitectura propuesta no busca imponer rigidez, sino evitar el desorden progresivo.

Si se respeta esta disciplina, el sistema no solo escalará técnicamente, sino también organizativamente.

FIN DEFINITIVO DEL ANEXO III